

Author: Kyle

Goal: Understand the theory behind extended kalman filters

Extended Kalman Filter: closed loop recursive algorithm used to estimate the the location of a system with noisy or incomplete data in nonlinear motion. (It is the nonlinear version of a Kalman filter)

Motivation: By using sensor fusion, it filters noisy sensor data to estimate the true position of the system (our robot).

Variables (assume linear algebra notation (ex: T is transpose and I is identity matrix)):

$\vec{x}$ = position vector/state vector (n=0 is starting position)

P_n = how much uncertainty (officially the covariance matrix)

$$P_n = \begin{bmatrix} \text{Var}(x) & \text{Cov}(x, y) & \text{Cov}(x, \theta) \\ \text{Cov}(x, y) & \text{Var}(y) & \text{Cov}(y, \theta) \\ \text{Cov}(x, \theta) & \text{Cov}(y, \theta) & \text{Var}(\theta) \end{bmatrix} \xrightarrow[\text{with}]{\text{We will initialize P}} P_0 = \begin{bmatrix} \sigma_{x,0}^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix}$$

Q = process noise covariance (has values for how much we trust the sensor data)

$$Q = \begin{bmatrix} \sigma_{x,\text{proc}}^2 & 0 & 0 \\ 0 & \sigma_{y,\text{proc}}^2 & 0 \\ 0 & 0 & \sigma_{\theta,\text{proc}}^2 \end{bmatrix}$$

K_n = kalman gain (how much we should weigh/trust the new sensor readings)

H = measurement jacobian (matrix with partial derivatives with respect to the variables in the state vector)

F = Process model jacobian (matrix with partial derivatives of the process model/ with respect to the state variables)

R = Noise covariance (matrix of containing values of how noisy the sensors are)

z = innovation (the difference between the prediction and sensor value)

Author: Kyle

Explaining Steps:**Step 1:** Initialize position x . So for us:

$$x_0 = [x, y, \theta]$$

Step 2: Prediction Step

- We first predict where the robot has moved on the field through its sensors (this is also known as process model). For us, this is:

$$\hat{x}_{n,\text{prior}} = F \hat{x}_{n-1,\text{update}} \longrightarrow \begin{array}{l} \circ x += \Delta\text{forward} * \cos(\text{heading}) - \\ \quad \Delta\text{horizontal} * \sin(\text{heading}) \\ \circ y += \Delta\text{forward} * \sin(\text{heading}) + \\ \quad \Delta\text{horizontal} * \cos(\text{heading}) \\ \circ \text{We don't change theta since it} \\ \quad \text{depends on the IMU (inertial} \\ \quad \text{sensor), not motor encoder} \end{array}$$

- We then compute the Jacobian to linearize the movement:

$$f(x, y, \theta) = \begin{bmatrix} x + v \cos(\theta) - h \sin(\theta) \\ y + v \sin(\theta) + h \cos(\theta) \\ \theta \end{bmatrix} \longrightarrow F_x = \frac{\partial f}{\partial [x, y, \theta]} = \begin{bmatrix} 1 & 0 & -v \sin(\theta) - h \cos(\theta) \\ 0 & 1 & v \cos(\theta) - h \sin(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

- Calculate covariance matrix prior to the update step:

$$P_{n,\text{prior}} = F P_{n-1,\text{update}} F^T + Q \longrightarrow \text{Updates uncertainty}$$

Author: Kyle

Step 3: Update IMU (inertial sensor correction). Note that we only correct the IMU because that is the only independent independent sensor independent/unused sensor as the tracking wheels and wheel encoders are used in the prediction step, so there is nothing to correct x and y position. For this:

- Calculate innovate:

Innovation = imu heading - predicted heading

- Calculate innovate covariance (defined as S):

$$S = HP_{n,\text{prior}}H^T + R_{\text{measure}}$$

- Calculate Kalman Gain:

$$K_n = P_{n,\text{prior}}H^T (HP_{n,\text{prior}}H^T + R_{\text{measure}})^{-1}$$



Note: we already calculated the parenthesis as S

$$K_n = P_{n,\text{prior}}H^T (S)^{-1} K_n$$

- Update state vector (where the robot is):

$$\hat{x}_{n,\text{update}} = \hat{x}_{n,\text{prior}} + K_n [z_{n,\text{measure}} - H\hat{x}_{n,\text{prior}}]$$

- Calculate the updated covariance matrix:

$$P_{n,\text{update}} = (I - K_n H) P_{n,\text{prior}}$$

Author: Kyle

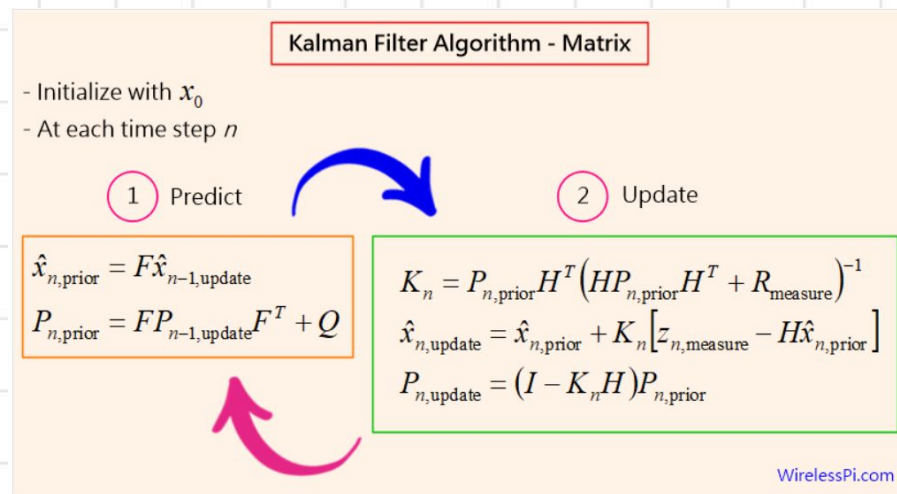


Fig 1.1 flow chart overview of the extended kalman filter. Image courtesy of Chaudhari (citation below).

Summary: The extended kalman filter first initializes where it is and how reliable the sensors are and then uses those sensor data to continuously predict and update where the robot is on the field.

Citation:

Chaudhari, Qasim. "The Extended Kalman Filter (EKF)." Wireless Pi, 17 May 2025, wirelesspi.com/the-extended-kalman-filter-ekf/.